

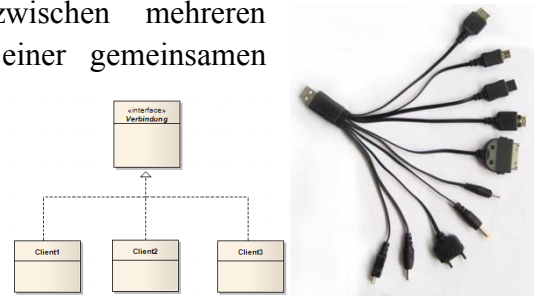
Interfaces

Interfaces und implements → class Client1 implements Verbindung {}
public interface Verbindung {}

Oft besteht die Notwendigkeit die Kommunikation zwischen mehreren verschiedenartigen Elementen zu vereinheitlichen und in einer gemeinsamen Schnittstelle festzulegen, damit sichergestellt ist, dass gewisse Funktionen immer gleich genutzt werden können.

Deshalb **müssen** alle Methoden des Interfaces auch in den implementierenden Klassen definiert werden.

→ Es dürfen auch weitere Methoden implementiert werden



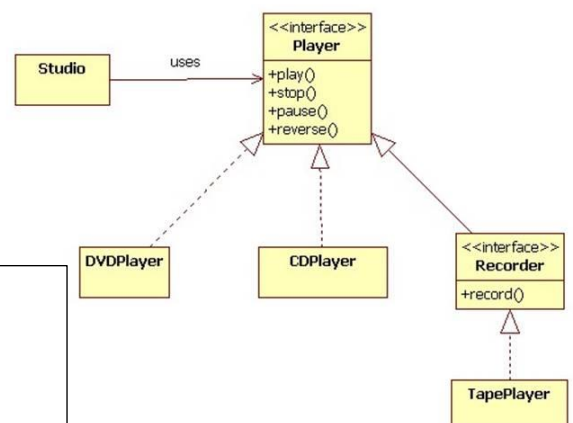
Vergleichbar ist dies mit einem Vertrag:

- Es können feste **Konstanten** definiert werden, z.B.
 - o Maximaler CO2 Ausstoß in Tonnen pro Jahr und Kopf: MAX_VALUE = 2 (Konstanten werden in ALL_CAPS geschrieben)
- Es wird festgesetzt, welche Regeln eingehalten bzw. welche **Funktionen bereitgestellt** werden müssen. Theoretisches Beispiel:
 - o EU-Gesetz – es muss in jedem Land Gesetze geben, um den CO2 Ausstoß unter MAX_VALUE Tonnen pro Jahr und Kopf zu senken
 - o mögliches Landesgesetz – Ausstoß von maximal 1.5t pro Jahr und Kopf
 - o nicht möglich: Ausstoß von maximal 5t pro Jahr und Kopf

→ Es wird also eine Mindestanforderung gestellt.

Beispiel: nahezu alle Autos haben die gleichen grundsätzlichen Steuerelemente (Lenkrad, Bremse, Gaspedal, Hupe, Schaltstange), die sie zur Verfügung stellen müssen. Die Modelle und deren genaues Verhalten beim Benutzen dieser Funktionen unterscheiden sich jedoch je nach Leistung oder ob sie über ABS und Servolenkung verfügen.

Im Klassendiagramm wird die **implementiert-Beziehung** ebenso wie die ist-eine-Art-von-Beziehung mittels eines unausgefüllten Dreiecks als Spitze des Pfeils dargestellt. Der Strich des Pfeils wird gestrichelt umgesetzt.



Beachte:

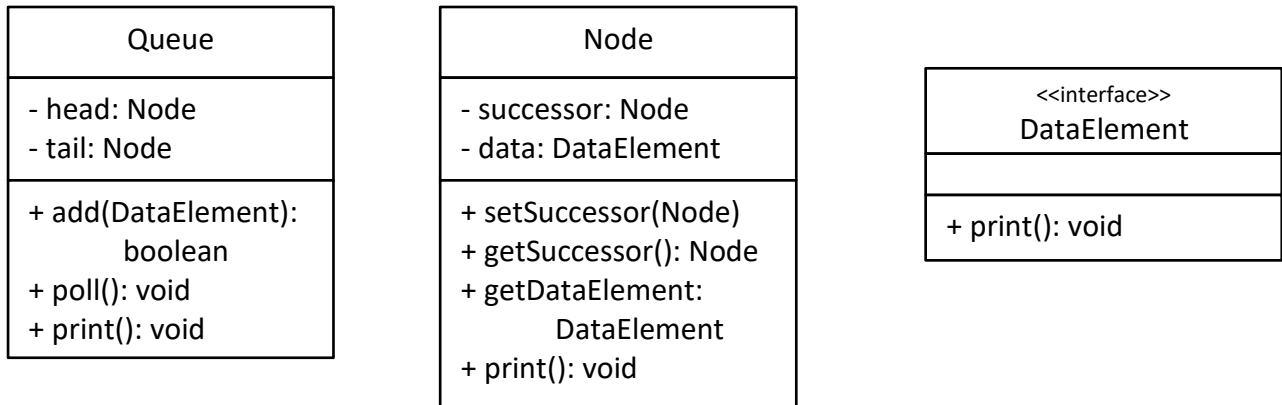
- In Interfaces sind alle Attribute implizit immer `public static final`
- Es werden nur **Methodenköpfe** deklariert, aber kein Methodenrumpf definiert. Methoden sind implizit `public`.
- Eine Klasse kann **mehrere** Interfaces implementieren!

Queue: Step-by-Step Guide

Umsetzung: Schlange von Kunden

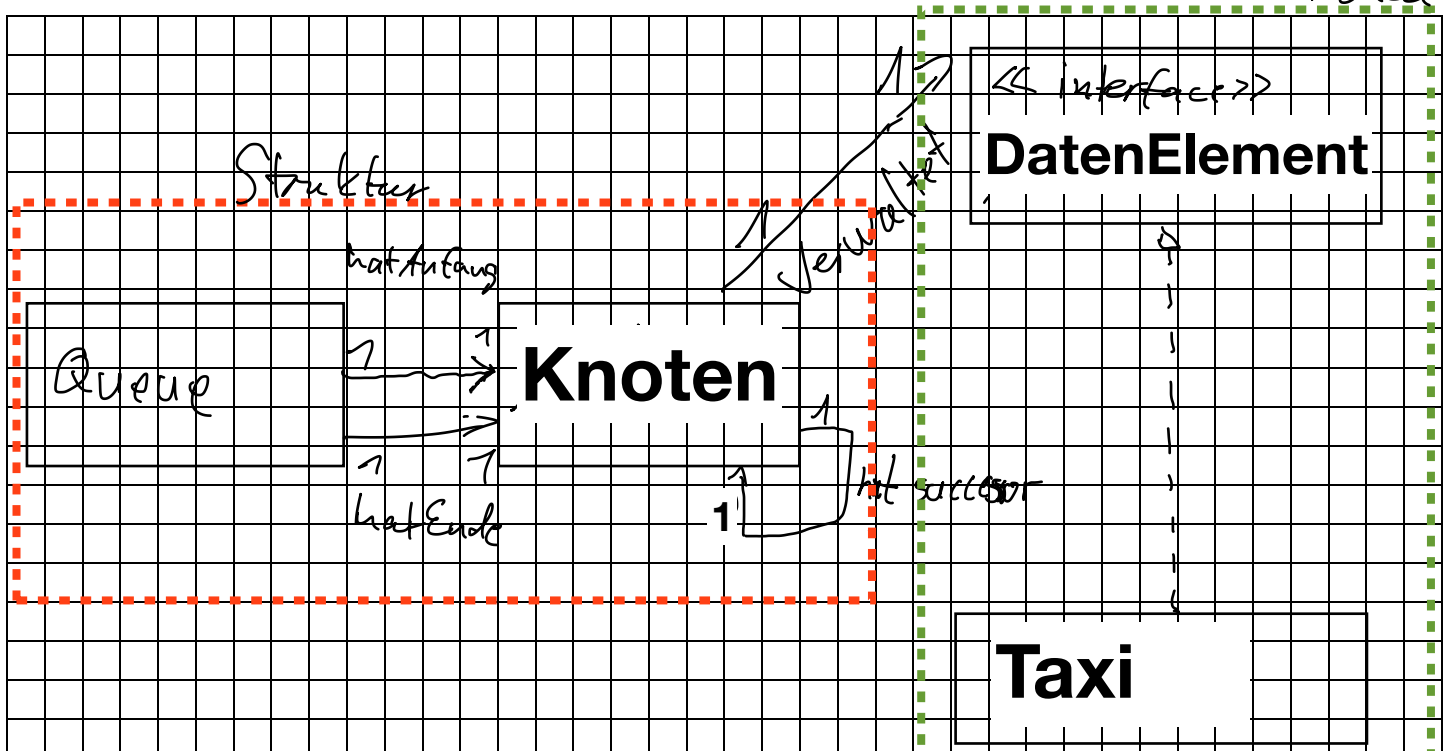
Im Folgenden wird eine `Customerqueue` programmiert – beginnend bei einem neuen Projekt.

- Erstellen Sie ein neues BlueJ-Projekt `Customerqueue`.
- Erstellen Sie die Klassen für **Queue** und **Node** sowie das Interface **DataElement**.



Die Methoden zur Informationsausgabe (→ print) sollen später die Informationen aus dem konkreten Datenelement (**Customer**) ausgeben. Die Queue gibt dies für all seine Knoten aus.

- Erstellen Sie eine Klasse **Customer** die das Interface `DataElement` implementiert. Zeichnen Sie ein vereinfachtes Klassendiagramm (nur Klassennamen) inklusive aller Beziehungen und Multiplizitäten. Zeichnen Sie außerdem ein simples Objektdiagramm mit der Queue, drei Nodes und den Datenobjekten (Customer). Stellen Sie die Verweise zwischen den Objekten dar. *Zukunft*



- Erstellen sie eine Klasse `Main` in der eine Reihe an Kunden die Schlange betreten und wieder verlassen. Geben Sie dabei je eine passende Ausgabe auf der Konsole aus. Lassen Sie in regelmäßigen Abständen auch die gesamte Queue ausgeben.

Durch die Trennung von Struktur und Inhalt erhöhen wir die Kohäsion und verringern die Kopplung. Dies sind essentielle Ideen in der Programmierung, um den Code modularer und damit leichter wiederverwendbarer zu gestalten.

Kohäsion beschreibt, wie gut eine Programmeinheit eine logische Aufgabe oder Einheit abbildet. In einem System mit **starker Kohäsion** ist jede **Programmeinheit** (eine Methode, eine Klasse oder ein Modul) verantwortlich für genau eine wohldefinierte Aufgabe oder Einheit.

Kopplung beschreibt die Verknüpfung von verschiedenen Systemen, Anwendungen oder Softwaremodulen sowie ein Maß, das die Stärke dieser Verknüpfung bzw. der daraus resultierenden Abhängigkeit beschreibt. Die einzelnen Module sollen dabei **möglichst unabhängig** sein. Man spricht von loser **Kopplung**.